1-D Transient Heat Conduction in a Flat Plate

Explicit (FTCS) Finite-Difference Solution in C

Course Project 7 – AM5630 Foundation of Computational Fluid Dynamics (Jul–Nov 2024)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras
Faculty: Prof. Arul Prakash

Revised report, October 2026

Abstract

The transient temperature field in a one-dimensional plate with a sudden step in wall temperature is computed with the explicit forward-time / central-space (FTCS) finite-difference scheme implemented in C. The solution is obtained for three time steps ($\Delta t = 0.1$, 0.01, 0.001 s) and compared with the exact Fourier-series solution. The original code was reviewed; three defects (an integer-typed length, a one-step offset in the time labels, and insufficient output precision) were corrected in the C code, and an in-place update in the MATLAB plotting script that silently turned the explicit scheme into a Gauss–Seidel-type sweep was fixed. The corrected solver reproduces the analytical profile to within 5.3×10^{-3} (maximum absolute error) at $t = 10$ s, the error being dominated by the spatial truncation term $\mathcal{O}(\Delta x^2)$. A run at $\gamma = 0.625$ confirms the von Neumann stability limit $\gamma \leq 1/2$.

Contents

1 Problem definition **2**

2 Numerical formulation **2**

2.1 Discretisation . . . 2
2.2 Stability . . . 2
2.3 Modified equation and consistency . . . 2
2.4 Analytical reference solution . . . 2
2.5 Algorithm and flowchart . . . 3

3 Code review and corrections **3**

4 Results **4**

4.1 Temperature profiles . . . 4
4.2 Accuracy . . . 5
4.3 Accuracy study: grid refinement and the effect of γ . . . 5
4.4 Comparison with the original plots . . . 6
4.5 Stability demonstration . . . 7

5 Conclusions **7**

A Source code (corrected) 7

1 Problem definition

A plate of length $L = 100$ (length units of the original code; written as cm) is initially at $T = 0$. At $t = 0^+$ the left face is raised to $T = 1$ and the right face is held at $T = 0$. Heat transfer is by conduction only, with constant diffusivity $\alpha = 1$.

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \qquad 0 < x < L,\ t > 0, \tag{1}$$
$$T(x,0) = 0, \qquad T(0,t) = 1, \qquad T(L,t) = 0.$$

(The original report titled the problem "heat convection"; Eq. (1) is the pure-diffusion, i.e. conduction, equation and that is what the code solves.)

2 Numerical formulation

2.1 Discretisation

The domain is divided into $N_x = 51$ nodes, $\Delta x = L/(N_x - 1) = 2$. Forward differencing in time and second-order central differencing in space give

$$T_i^{n+1} = \gamma\, T_{i-1}^n + (1 - 2\gamma)\, T_i^n + \gamma\, T_{i+1}^n, \qquad \gamma = \frac{\alpha\, \Delta t}{\Delta x^2}, \tag{2}$$

with local truncation error $\mathcal{O}(\Delta t, \Delta x^2)$. All right-hand-side values belong to the *old* time level n, which is what makes the scheme explicit.

2.2 Stability

Substituting a Fourier mode $T_i^n = G^n e^{\mathrm{i}ki\Delta x}$ into (2) gives the amplification factor $G = 1 - 4\gamma \sin^2(k\Delta x/2)$. $|G| \le 1$ for all wavenumbers requires

$$\gamma = \frac{\alpha \Delta t}{\Delta x^2} \le \frac{1}{2}. \tag{3}$$

For $\Delta x = 2$ the limit is $\Delta t \le 2$ s; the three production runs use $\gamma = 2.5\times10^{-2}$, 2.5×10^{-3}, 2.5×10^{-4}, and an extra run at $\Delta t = 2.5$ s ($\gamma = 0.625$) is used to demonstrate the instability.

2.3 Modified equation and consistency

Expanding (2) in Taylor series about (x_i, t^n) and eliminating the time derivatives with the PDE gives the equation that the scheme actually solves,

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} + \frac{\alpha \Delta x^2}{12}\,(1 - 6\gamma)\, \frac{\partial^4 T}{\partial x^4} + \mathcal{O}(\Delta x^4). \tag{4}$$

The scheme is consistent and second-order in space at fixed γ; the leading error term changes sign at $\gamma = 1/6$. Equivalently, the per-step decay of a Fourier mode, $\ln G = -\gamma\theta^2 + \gamma\theta^4(1-6\gamma)/12 + \ldots$ with $\theta = k\Delta x$, matches the exact $-\gamma\theta^2$ to fourth order only when $\gamma = 1/6$.

2.4 Analytical reference solution

Separation of variables gives the exact solution used for verification:

$$T(x,t) = 1 - \frac{x}{L} - \frac{2}{\pi} \sum_{n=1}^{\infty} \frac{1}{n} \sin\!\Big(\frac{n\pi x}{L}\Big) \exp\!\Big(-\frac{n^2 \pi^2 \alpha t}{L^2}\Big), \tag{5}$$

evaluated with 400 terms.

2.5 Algorithm and flowchart

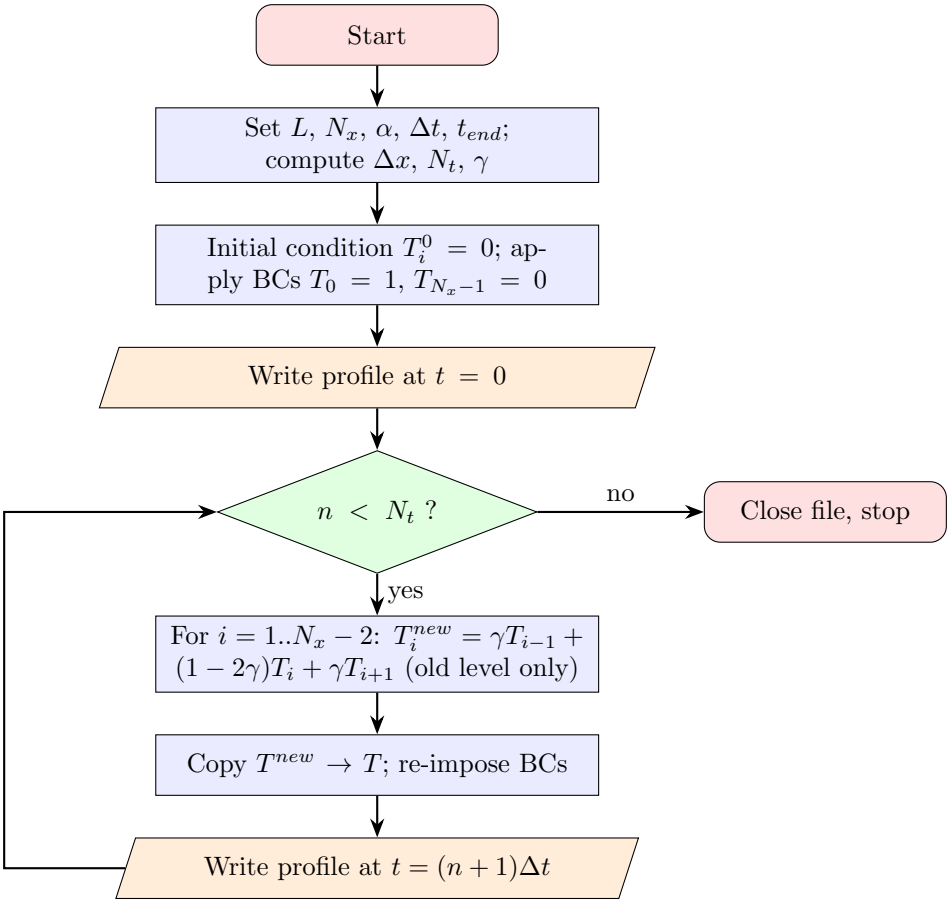


Figure 1: Flowchart of the explicit FTCS solver (`code/assignment_1.c`).

3 Code review and corrections

The original code was kept unchanged except for the defects below (every change is marked `[FIX]` in the source; the untouched originals are in `original/`).

#	Location	Problem	Correction
1	C: `int L = 100;`	$\Delta x = L/(N_x - 1)$ is computed in integer arithmetic. It happens to be exact for $L = 100$, but the version in the submitted report ($L = 1$) gives $\Delta x = 0$ and $\gamma = \infty$.	Declared `double L = 100.0`.
2	C: time loop	The loop started at $j = 0$ and updated the field *before* writing it under the label $t = j\Delta t$, so every stored profile was one step later than its label and $t = 0$ was never written.	Write the initial condition first; loop $j = 1..N_t$ and label the state after j steps $t = j\Delta t$.
3	C: output	Profiles were written with 2 decimals, hiding the early-time solution ($T < 0.005$ printed as 0.00).	6 decimals.
4	MATLAB: interior update	`grid(i)` was overwritten in place, so `grid(i-1)` on the right-hand side was already the new value. This is a Gauss–Seidel-type sweep, not Eq. (2); the same one-step label offset (fix 2) was also present. The plots in the original report came from this script.	Use a copy `Told` of the previous level on the right-hand side.

Table 1: Defects found and corrected.

The numerical method, grid, parameters and output format were not otherwise changed. The console printing of every node at every step was left in place; `run.sh` redirects it to `/dev/null`.

4 Results

4.1 Temperature profiles

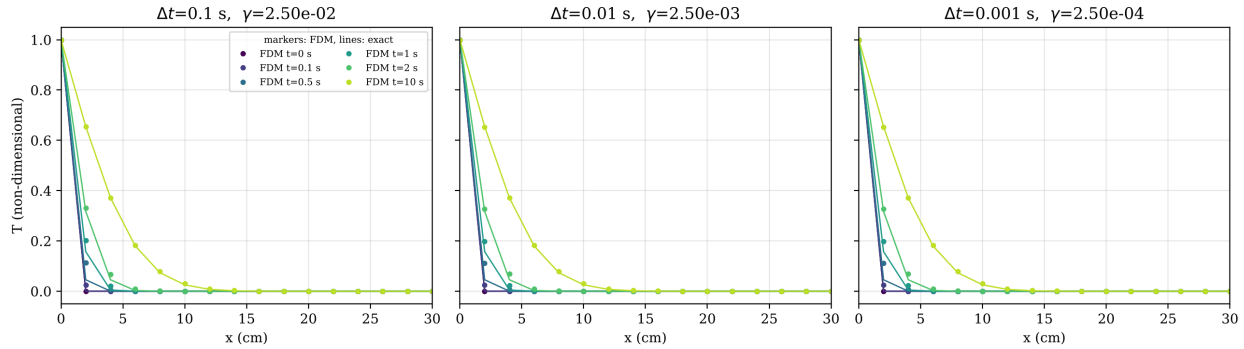
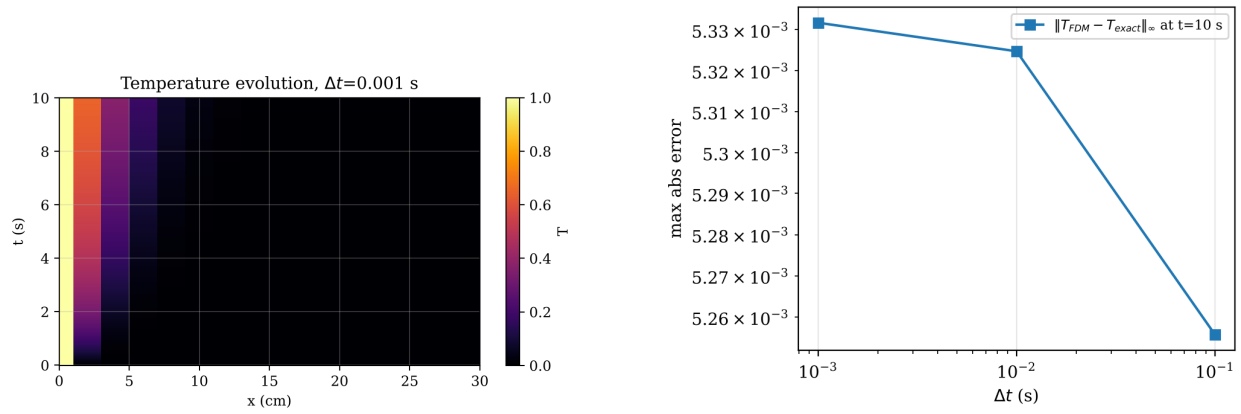


Figure 2: FDM profiles (markers) against the exact solution (lines) at $t = 0, 0.1, 0.5, 1, 2, 10$ s for the three time steps. Only $0 \le x \le 30$ is shown; the field is zero beyond.

The thermal front penetrates a distance of order $\sqrt{\alpha t}$ (≈ 3.2 at $t = 10$ s), so at the final time the disturbance has reached only $\sim 15\%$ of the plate and the right boundary plays no role. The three time steps give visually identical profiles, and all agree with the exact solution; at $x = 2$, $t = 10$ s the FDM gives $T = 0.6525$ against $T = 0.6547$ exactly.



(a) Space–time map, $\Delta t = 0.001$ s.

(b) Maximum error at $t = 10$ s.

Figure 3: Evolution of the field and dependence of the error on Δt.

4.2 Accuracy

Δt (s)	γ	N_t	$\max \lvert T_{FDM} - T_{exact} \rvert$ at $t = 10$ s
0.1	2.5×10^{-2}	100	5.26×10^{-3}
0.01	2.5×10^{-3}	1 000	5.33×10^{-3}
0.001	2.5×10^{-4}	10 000	5.33×10^{-3}

Table 2: Error against the exact solution.

The error is essentially independent of Δt: because $\gamma \ll 1/2$ in all runs the temporal error $\mathcal{O}(\Delta t)$ is negligible and the remaining error is the spatial truncation term $\mathcal{O}(\Delta x^2)$ with $\Delta x = 2$. Refining the grid, not the time step, is the way to improve accuracy for this set-up; the largest stable time step ($\gamma \to 1/2$) would therefore be the most economical choice.

4.3 Accuracy study: grid refinement and the effect of γ

Two additional sets of runs of the same program (only `CX`, `dt` and the end time substituted, `run.sh`) quantify the accuracy.

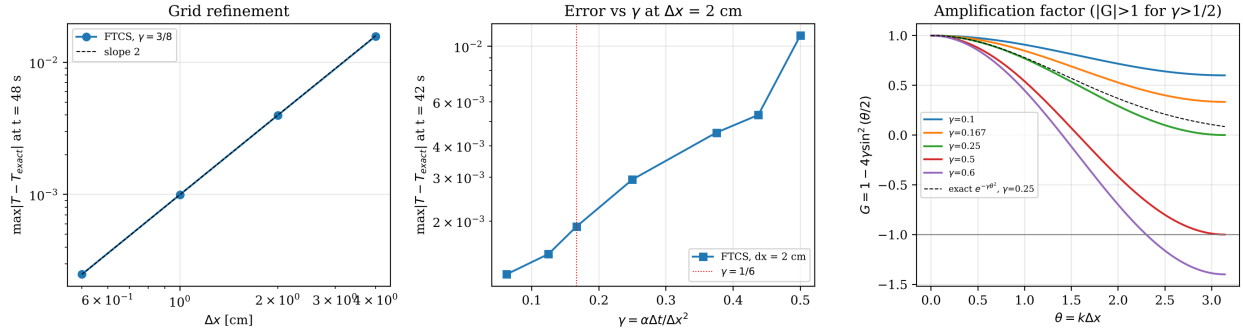


Figure 4: Left: maximum error at $t = 48$ s for $\Delta x = 4, 2, 1, 0.5$ cm at fixed $\gamma = 3/8$. Centre: error at $t = 42$ s against γ for $\Delta x = 2$ cm. Right: amplification factor of FTCS against the exact decay $e^{-\gamma\theta^2}$.

N_x	Δx [cm]	$\max \lvert T - T_{exact} \rvert$, $t = 48$ s	observed order
26	4	1.576×10^{-2}	–
51	2	3.973×10^{-3}	1.99
101	1	9.958×10^{-4}	2.00
201	0.5	2.488×10^{-4}	2.00

Table 3: Grid refinement at constant γ ($\Delta t \propto \Delta x^2$; `results/accuracy_study.txt`).

The observed order is 2.00, as predicted by the modified equation. At fixed $\Delta x = 2$ cm the error grows from 1.2×10^{-3} at $\gamma = 0.0625$ to 5.3×10^{-3} at $\gamma = 0.44$ and doubles again at the stability limit $\gamma = 1/2$, where the highest mode is no longer damped ($G(\pi) = -1$). The cancellation at $\gamma = 1/6$ predicted for smooth solutions does not appear as a minimum here, because the error at this resolution is dominated by the representation of the discontinuous start-up (the boundary jumps from 0 to 1 at $t = 0$), which no choice of γ removes. Practically, $\gamma \approx 0.1$–0.25 balances accuracy and cost.

4.4 Comparison with the original plots

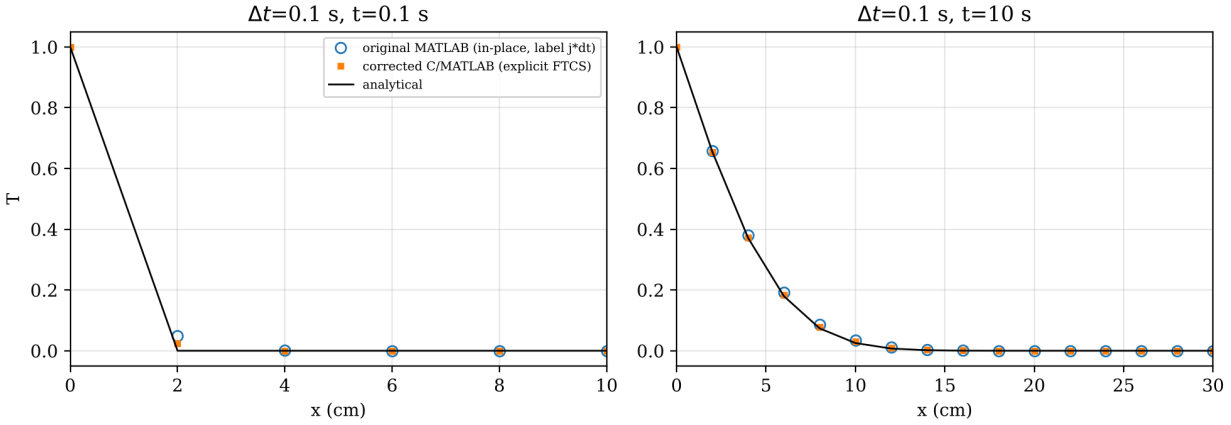


Figure 5: Original MATLAB result (in-place sweep, one-step label offset) versus the corrected explicit scheme and the exact solution, $\Delta t = 0.1$ s.

At $t = 10$ s the original and corrected results almost coincide, which is why the original plots looked correct. At early times the difference is visible: at $t = 0.1$ s the original plot shows $T(2) = 0.0487$ (two in-place updates) whereas the explicit scheme gives 0.025 after one step. The original plots are kept in `original/Original_Report_Plots.pdf`.

4.5 Stability demonstration

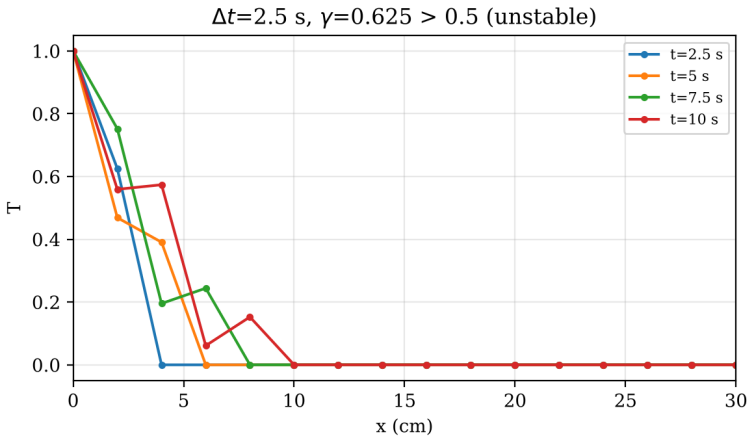


Figure 6: $\Delta t = 2.5$ s ($\gamma = 0.625$): the node-to-node (odd–even) oscillation characteristic of $|G| > 1$ appears immediately.

5 Conclusions

- The explicit FTCS scheme implemented in C reproduces the exact transient conduction solution to within 5.3×10^{-3} at $t = 10$ s for all three time steps.
- With $\gamma \ll 1/2$ the error is controlled by Δx; the time-step study shows temporal convergence is already saturated.
- The stability bound $\gamma \leq 1/2$ was confirmed numerically, and a four-grid refinement study confirmed second-order spatial convergence (observed order 2.00), in agreement with the modified-equation analysis.
- Four implementation defects were corrected without changing the method; the most significant was the in-place update in the MATLAB script, which had changed the scheme that produced the original plots.

Reproducing the results

`bash run.sh` compiles `code/assignment_1.c` for each Δt, runs it, and calls `code/plot_results.py` (Python 3, NumPy, Matplotlib). It then runs the accuracy studies (`code/study.py`). Outputs go to `results/`.

A Source code (corrected)

```c
// 1D heat convection Equation

#include <stdio.h>
#include <math.h>
#define CX 51// Number of grid points

int main() {
    // Declare the variables
    double L = 100.0; // cm [FIX 1: was int -> integer division risk in dx = L/(CX-1)]
    double grid[CX];
    double alpha = 1; // m^2/s
    double dt = 0.001; // sec
```

```
    double T0 = 1.0; // Boundary condition at x=0 length
    double TL = 0.0; // Boundary condition at x=L length
    double TN[CX]; // Temperature at new grid points
    double t = 10.0; // Total time for simulation
    int DX,DT; // for output file notation
    // Cell size calculation
    double dx; // Size of the cell in x-axis
    dx = L / (CX - 1);
    printf("dx: %lf\t", dx);
    printf("dt: %lf\t", dt);

    // Time steps calculations

    int Nt = (int)(t / dt); // Number of time steps, cast to int for loop control
    printf("Nt: %d\t", Nt);

    // Gamma Calculation
    double gamma = alpha * (dt / (dx * dx));
    printf("\ngamma: %f\t", gamma);

    // Initial condition: Set all temperatures to zero initially.
    for (int i = 0; i < CX; i++) {
        grid[i] = 0.0;
    }

    printf("\nInitial Temperature at grid Points:\n");
    for (int i = 0; i < CX; i++) {
        printf("grid[%d] = %.2f\n", i, grid[i]);
    }

    // Open a file for writing
    FILE *outputFile;
    outputFile = fopen("temperature_output.txt", "w");

    if (outputFile == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    // Write header to the file
    fprintf(outputFile, "DX\tDT\n"); // Header row


    // [FIX 2: write the initial condition at t = 0 before any update, and label each
    // stored profile with the time it actually represents (after j updates)]
    grid[0] = T0; grid[CX - 1] = TL;
    fprintf(outputFile, "Time: %.4f seconds\n", 0.0);
    for (int i = 0; i < CX; i++) fprintf(outputFile, "%d\t%.6f\n", i, grid[i]);
    fprintf(outputFile, "\n");

    // Time loop initializing
    for (int j = 1; j <= Nt; j++) {
        double Time = j * dt; // Calculating the run time (state after j steps)
        printf("\nTime: %.2f Seconds\n", Time);

        // Impose boundary conditions
        grid[0] = T0; // Left side Boundary condition
        grid[CX - 1] = TL; // Right side Boundary Condition

        for (int i = 1; i < CX - 1; i++) {
            TN[i] = (gamma * grid[i - 1]) + ((1 - (2 * gamma)) * grid[i]) + (gamma * grid[i + 1]);
            printf("\nTN[%d] = %.2f\n", i, TN[i]);
        }

        for (int i = 1; i < CX - 1; i++) {
            grid[i] = TN[i]; // Update temperature values for next iteration
        }

        // Write grid points and temperatures to the file after updating temperatures
        fprintf(outputFile, "Time: %.4f seconds\n", Time);
        for (int i = 0; i < CX; i++) {
      // fprintf(outputFile, "Grid%d: Temperature %.2f\n", i, grid[i]);
            fprintf(outputFile, "%d\t%.6f\n", i, grid[i]); // [FIX 3: 2 -> 6 decimals; 0.01 resolution hid the
                early-time profile]
        }
```

```
87          fprintf(outputFile, "\n"); // New line for separation between time steps
88      }
89
90      fclose(outputFile); // Close the file after writing
91
92      return 0;
93  }
```